



Week 5: Stacks



Dr. Sirasit Lochanachit



Today's Outline

1. What is a Stack?
2. Stack Abstract Data Type
3. Simple Array-Based Stack Implementation
4. Stack Applications



Linear Data Structures



- Items are ordered depending on how they are added or removed.
 - Arrays
 - Stacks
 - Queues

What is a Stack?



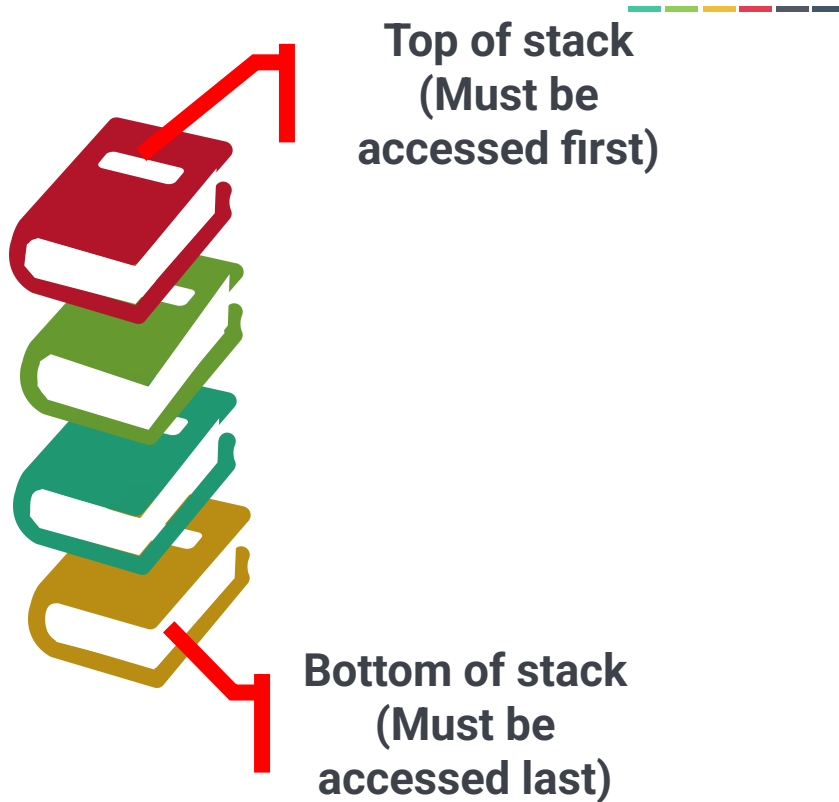
A stack is a collection of objects that are inserted and removed according to the ***last-in first-out (LIFO)*** principle [1].

The name “stack” is derived from the metaphor of a stack of plates in a plate dispenser.





Stack of Books



Stack Abstract Data Type



To create an instance of a stack, use `Stack()`.

Formally, there are 2 main operations of stacks:

- 1) `push(item)`
- 2) `pop()`



Stack of Books



Push
a new book on the top



Pop
a book from the top

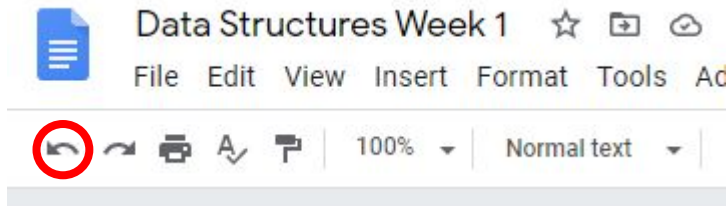
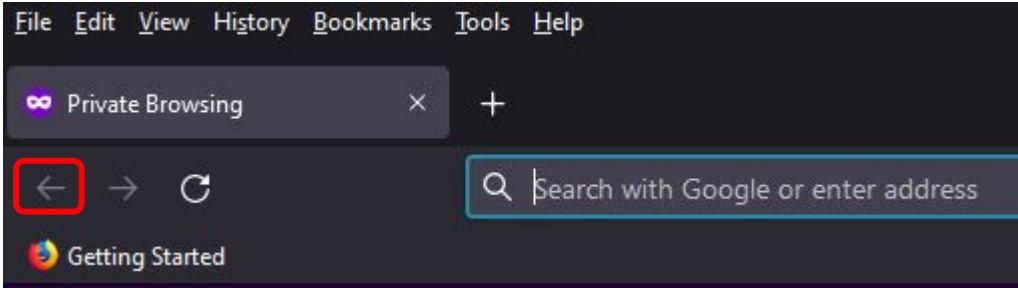


Stack Applications Example



Web Browser's
history of recently
visited sites.

The undo button.





Stack's Operations



Additional operations of stacks:

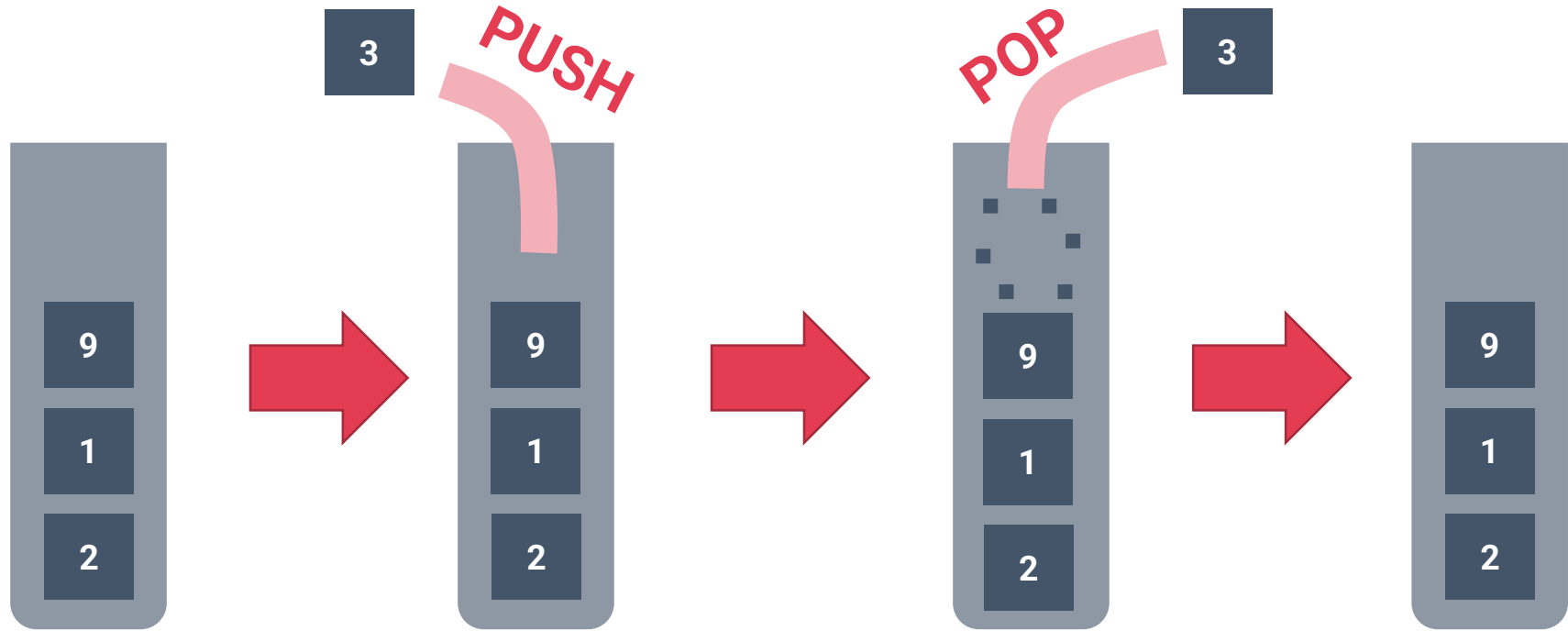
- 1) `top()`
- 2) `is_empty()`
- 3) `len(Stack)`

Stack Implementation



How to Implement a Stack?

Stack Implementation





Stack's Operation Example



Operation	Return Value
S.push(9)	
S.push(5)	
S.top()	
S.push(2)	
S.pop()	
S.is_empty()	
len(S)	

Stack



Asymptotic Performance



Operation	Running Time
S.push(element)	$O(1)$
S.pop()	$O(1)$
S.top()	$O(1)$
S.is_empty()	$O(1)$
len(S)	$O(1)$



Stack Implementation



How to Implement a Stack using a Python?

Stack Implementation

Create ArrayStack class and its methods

```
[2] 1 class ArrayStack:
      2     def __init__(self):
      3         """ Create an empty stack. """
      4         self._data = [] # Initiate a nonpublic list instance
      5
      6     def __len__(self):
      7         """ Return the number of elements in the stack. """
      8         return len(self._data)
      9
     10     def is_empty(self):
     11         """ Return True if the stack is empty. """
     12         return len(self._data) == 0
     13
     14     def push(self, element):
     15         """ Add an element to the top of the stack. """
     16         self._data.append(element) # new item stored at end of list
     17
```



Stack Implementation

```
18 def top(self):
19     """ Return (but do not remove) the element at the top of the stack.
20     Raise an exception if the stack is empty. """
21     if self.is_empty():
22         print('Stack is empty')
23         raise Empty('Stack is empty') # Calling subclass Empty
24     return self._data[-1] # the last item in the list
25
26 def pop(self):
27     """ Remove and return the element from the top of the stack.
28     Raise an exception if the stack is empty. """
29     if self.is_empty():
30         print('Stack is empty')
31         raise Exception('Stack is empty') # Alternate way to call subclass Empty
32     return self._data.pop() # remove last item from the list
33
```

```
[3] 1 class Empty(Exception):
    2     """ Error attempting to access an element from an empty container. """
    3     pass
```




Stack Applications

1) Simple Balanced Parentheses

Each opening symbol must match its corresponding closing symbol.

Parentheses: ()

Examples

Balanced:

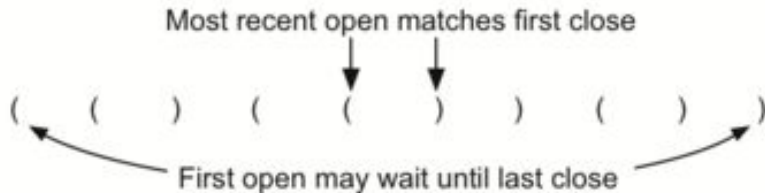
Unbalanced:



Simple Balanced Parentheses



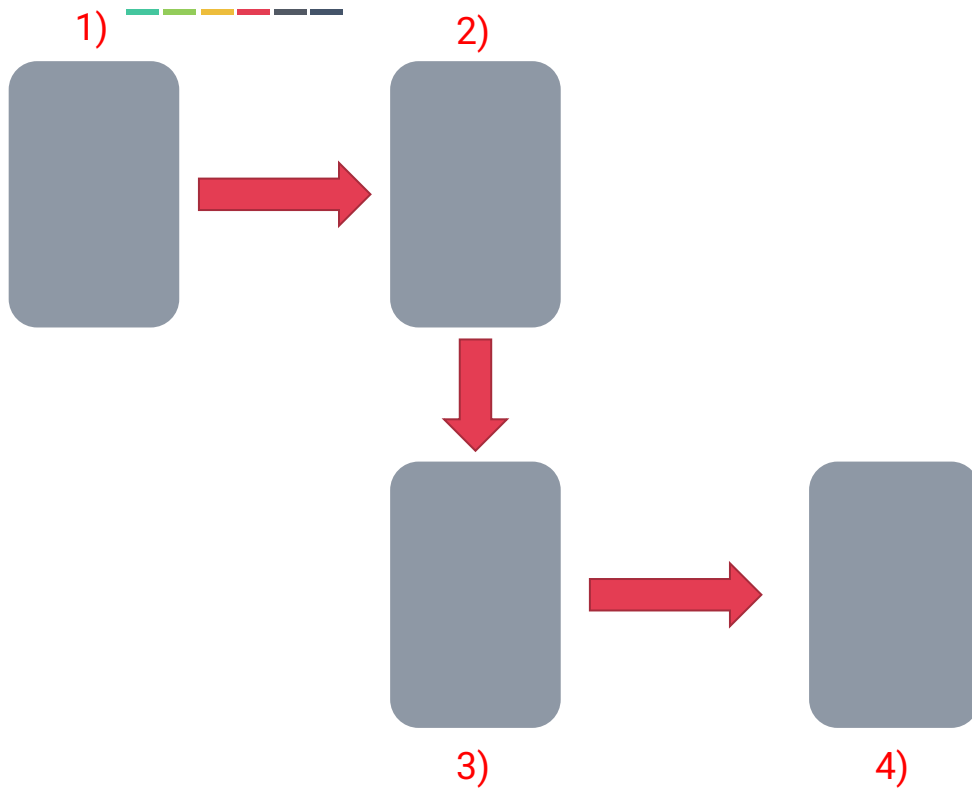
How to check balanced parentheses?





Simple Balanced Parentheses

Example: $(1 + x - (y + z))$



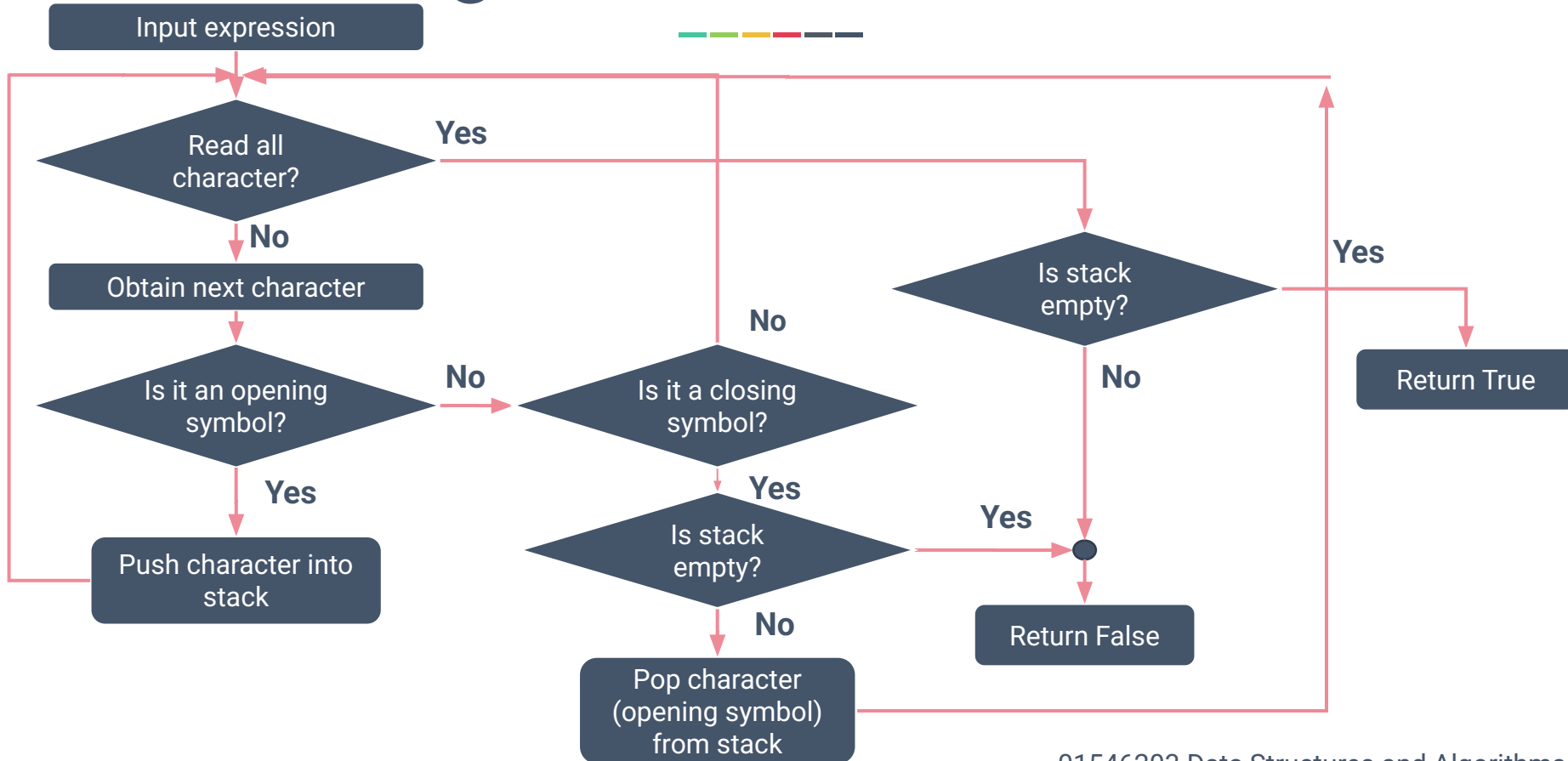


Balanced Parentheses Algorithm



IsBalanced(str):

Algorithm's Flowchart



Stack Applications



2) Balanced Grouping Symbols

Each opening symbol must match its corresponding closing symbol.

Parentheses: ()

Brackets: []

Braces: { }

Valid

()

[]

{ }

(([]))

{{ [] } }

([] { () [] }) [{ }]

Invalid

(

[[]

(]

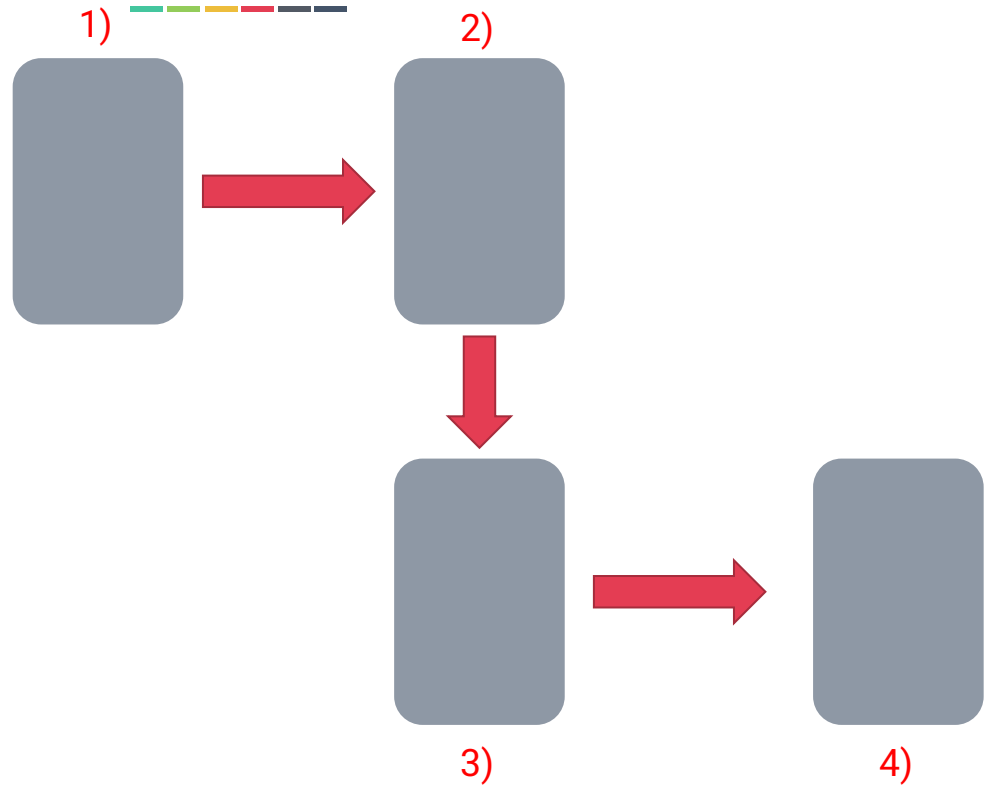
))

())

([] { () [] }) [{ }]

Balanced Grouping Symbols

Example: $(1 + x - [y + z])$





Balanced Grouping Symbols Algorithm



IsBalanced(str):

```
s = ArrayStack()
```

```
...
```

```
...
```

```
...
```

```
...
```

```
...
```

```
return s.is_empty()
```

This is your Lab 8-2 task!



Week 5: Queues



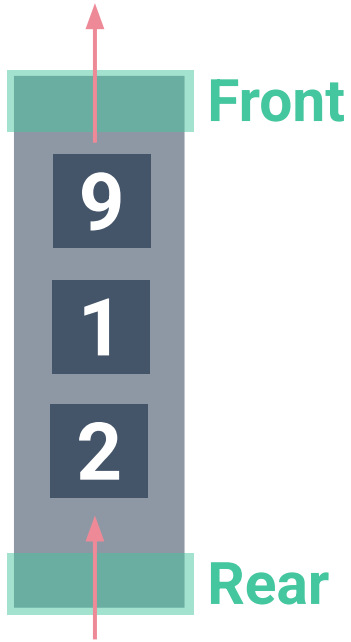
Dr. Sirasit Lochanachit



Today's Outline

1. What is a Queue?
2. Queue Abstract Data Type
3. Array-Based Queue Implementations
 - Simple Queue
 - Revised Queue
 - Circular Queue

Queues



A queue is a collection, which keeps objects in a sequence, that are inserted and removed according to the **first-in first-out** (FIFO) principle [1].



Queues



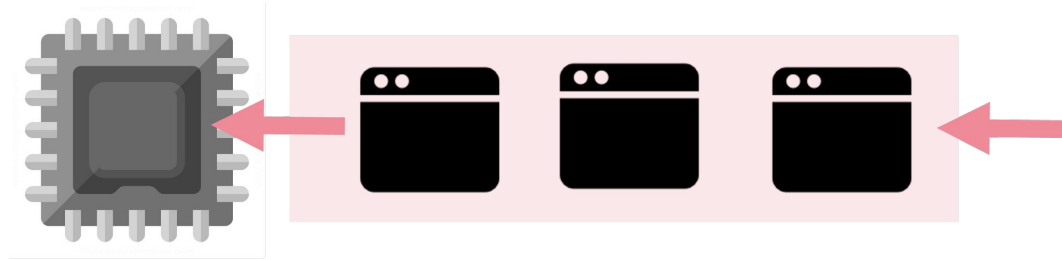
Real-life examples of queue:



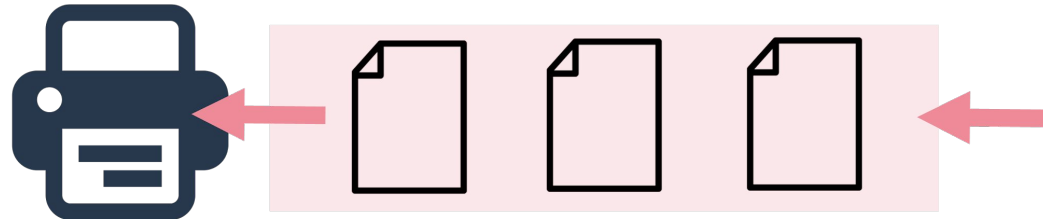
Queues



Examples of queue:



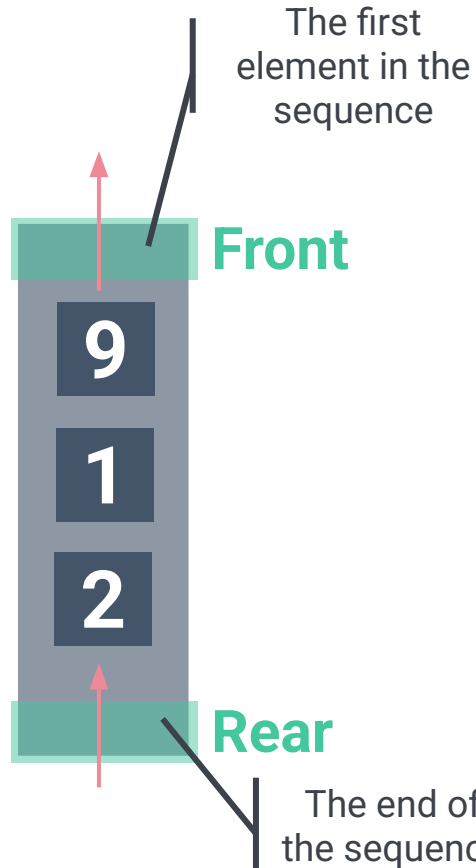
(i) a line of processes waiting to enter the CPU.



(ii) a line of paper waiting to print out.



The Queue Abstract Data Type



To create an instance of a queue, use `Queue()`.

Formally, there are 2 main operations for queues:

1) `enqueue(item)`

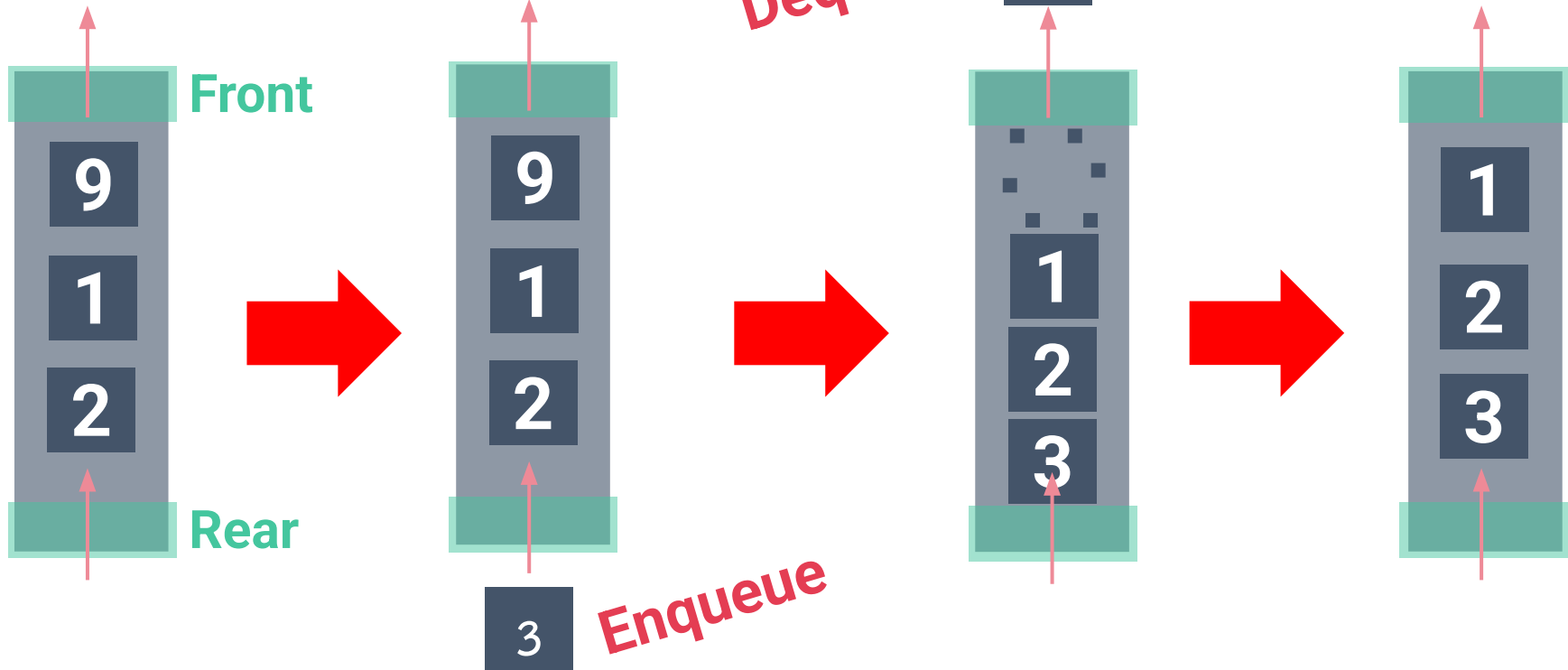
2) `dequeue()`



Queues



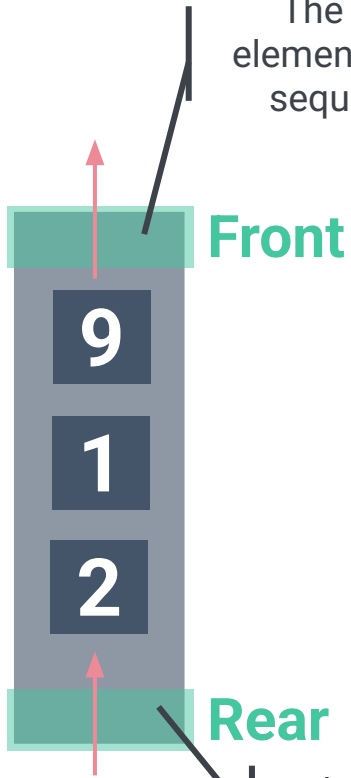
Dequeue



The Queue Abstract Data Type



The first
element in the
sequence



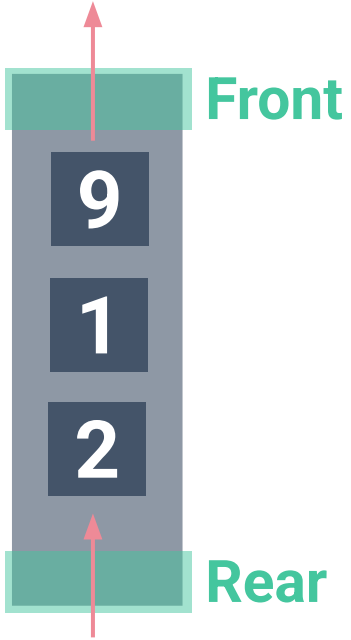
Additional operations for queues:

- 1) **first()**
- 2) **is_empty()**
- 3) **len(Queue)**

The end of
the sequence



Queues



How to Implement a Queue?



Operation Example



Operation	Return Value
Q.enqueue(9)	-
Q.enqueue(5)	-
Q.first()	
Q.enqueue(2)	-
Q.dequeue()	
Q.is_empty()	
len(Q)	



Queue
[first, ..., last]



Asymptotic Performance



Operation	Running Time
Q.enqueue(item)	
Q.dequeue()	
Q.first()	
Q.is_empty()	
len(Q)	



Simple Queue



Suppose a Python list is created.

- Enqueue - $O(?)$
- Dequeue - $O(?)$



Simple Queue



After item 9 is dequeued, the queue has to shift all the remaining items to the left.

Therefore, dequeue operation is $O(n)$



Alternative Simple Queue



Suppose a Python list is created.

- Enqueue - $O(?)$
- Dequeue - $O(?)$



Revised Queue



A better approach is to assign two main pointers to store indices that refer to:

the first element of the queue, which is called the **Front**,

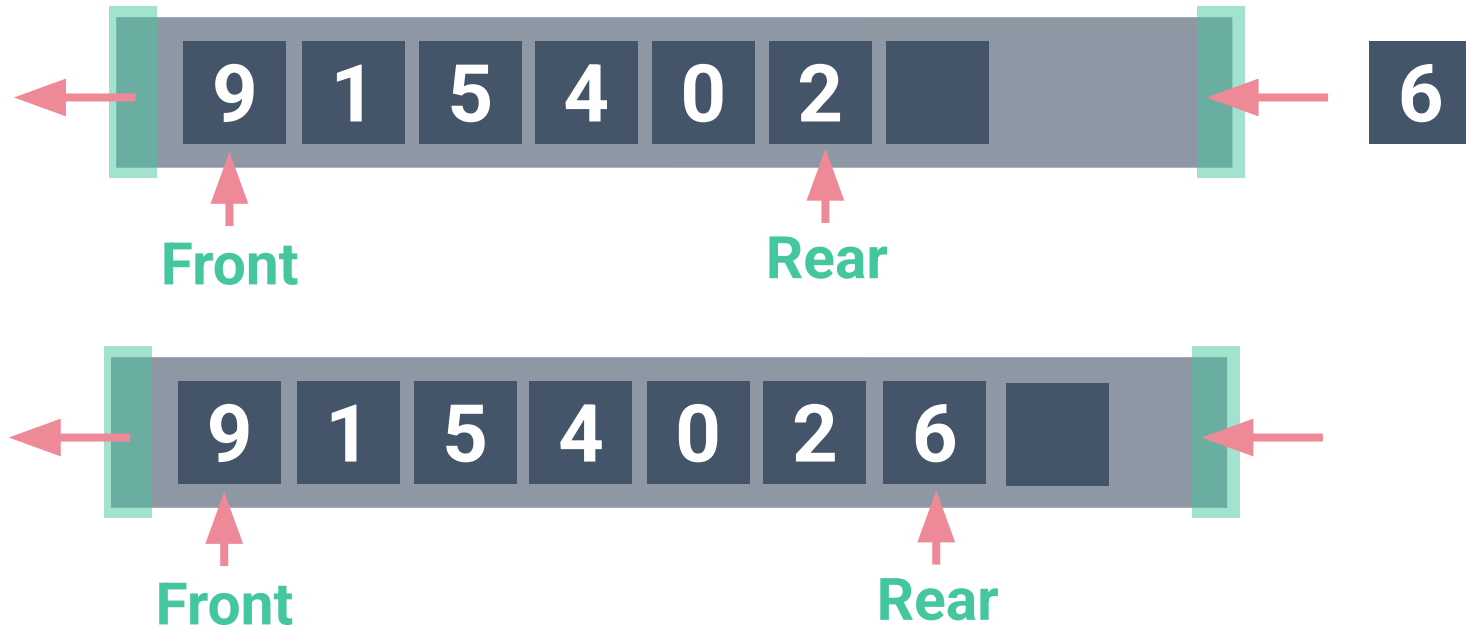
and the last element of the queue, which is known as the **Rear**.



Revised Queue



- Enqueue

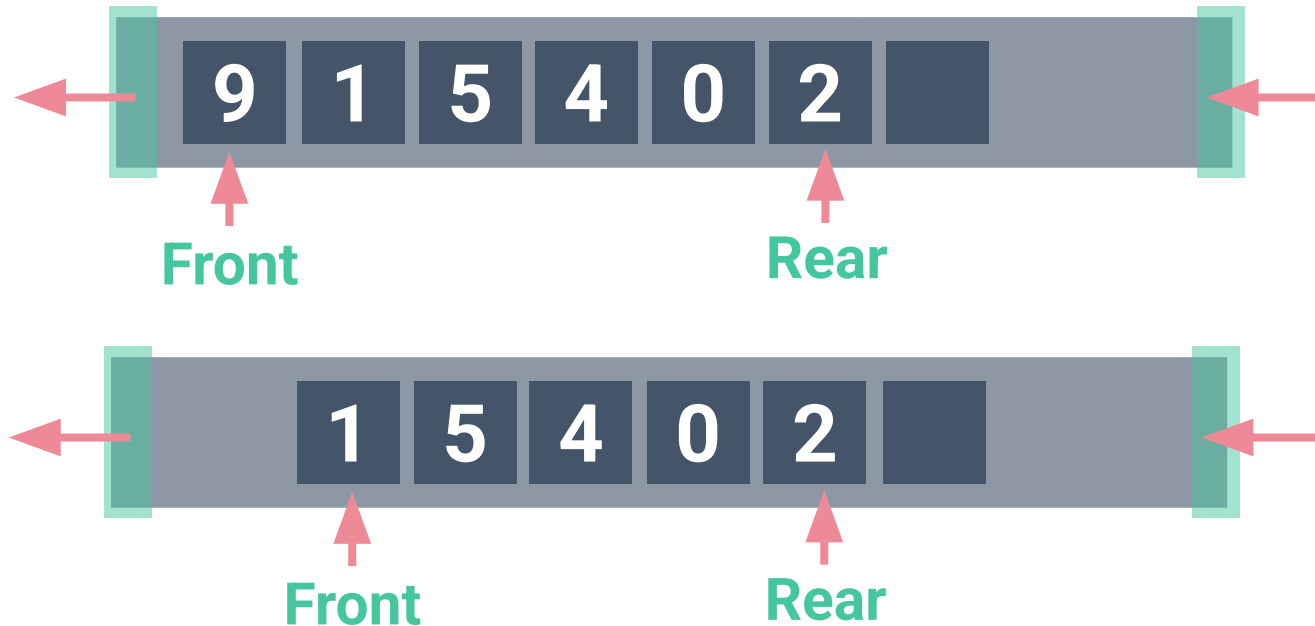




Revised Queue

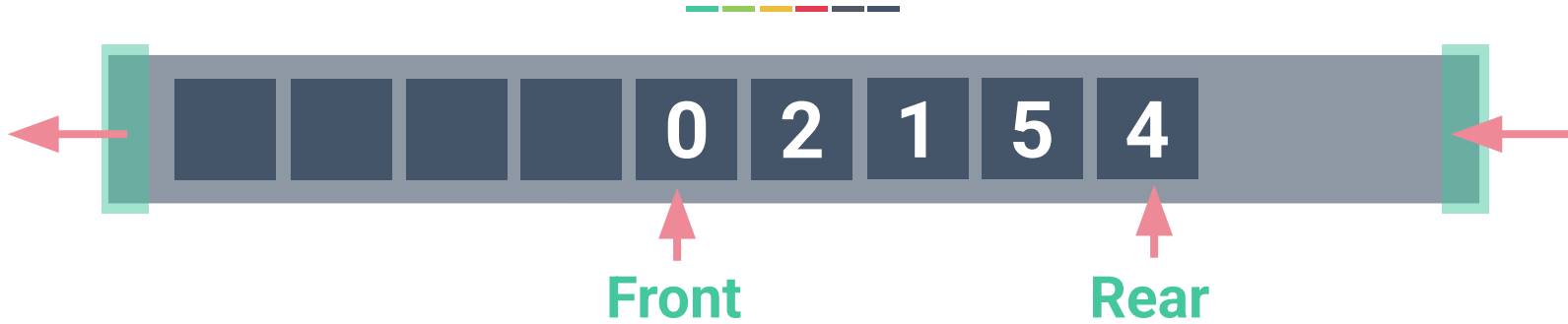


- **Dequeue**





Revised Queue



Drawback:

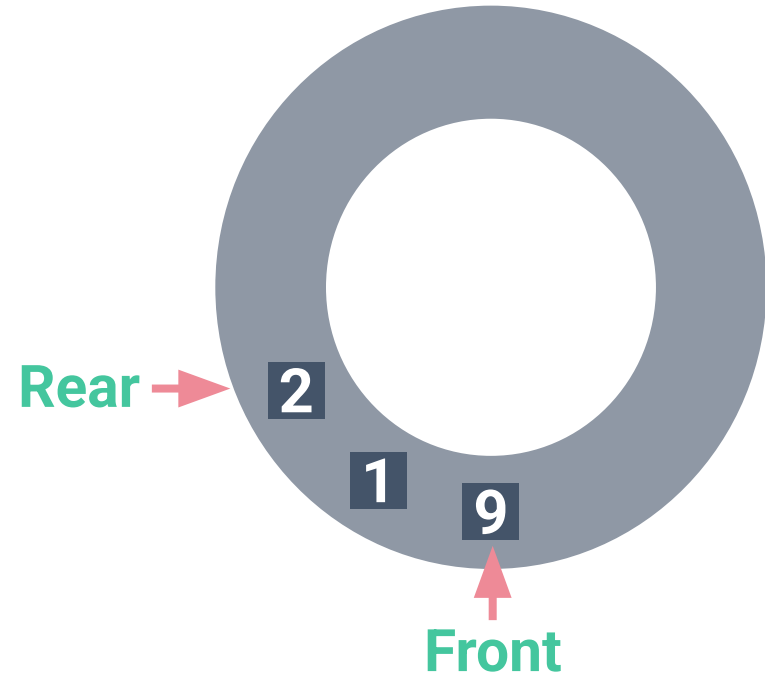
Types of Queue



- Queue



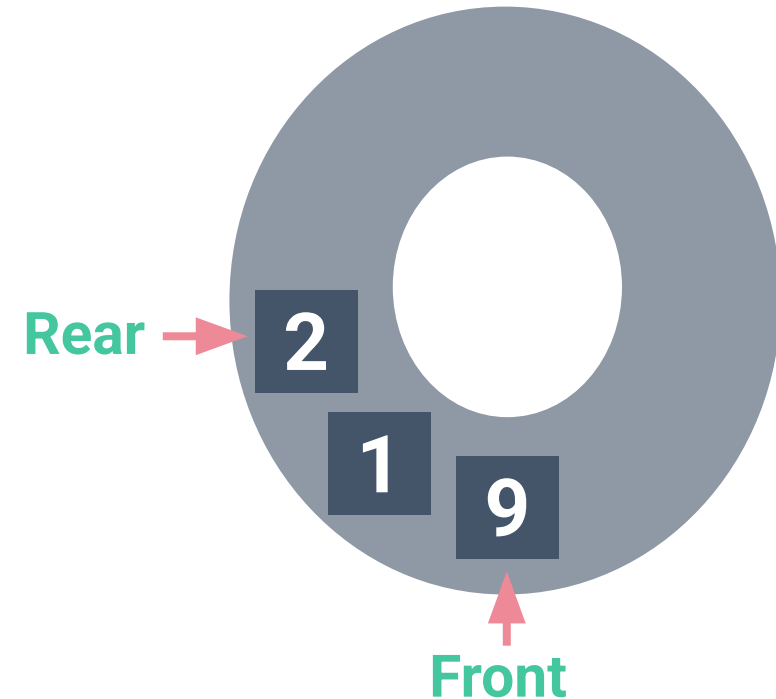
- Circular Queue



Circular Queue



In addition to allowing the pointers to shift rightward, the elements of the queue are wrap around at the end of an array.



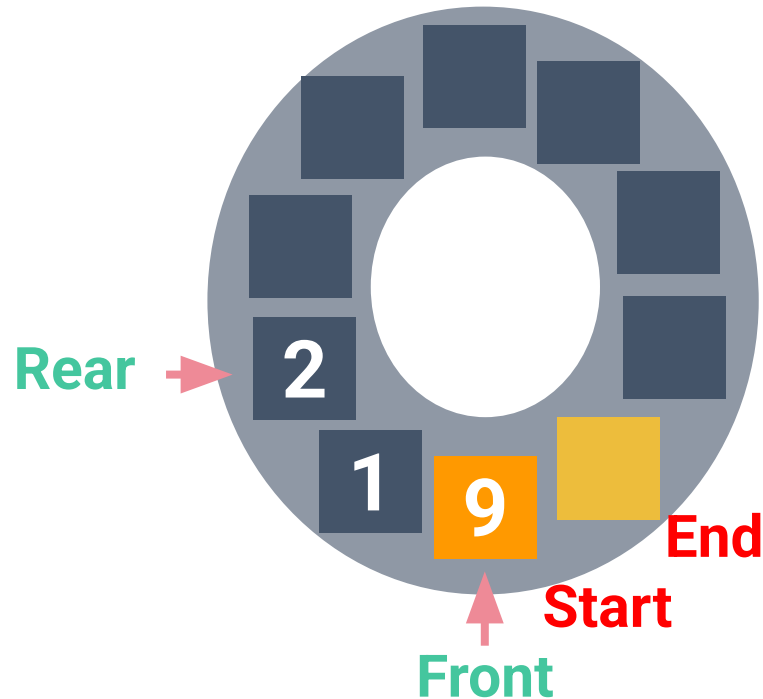


Circular Queue



Suppose a queue of length 10,

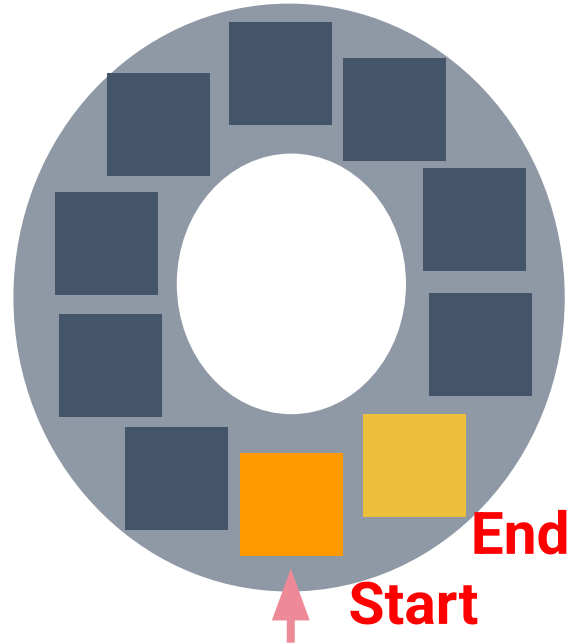
Circular Queue



Circular Queue

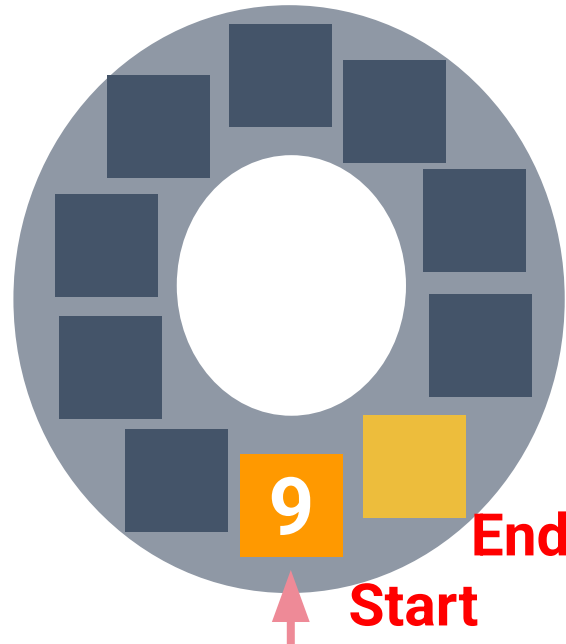


Empty Queue



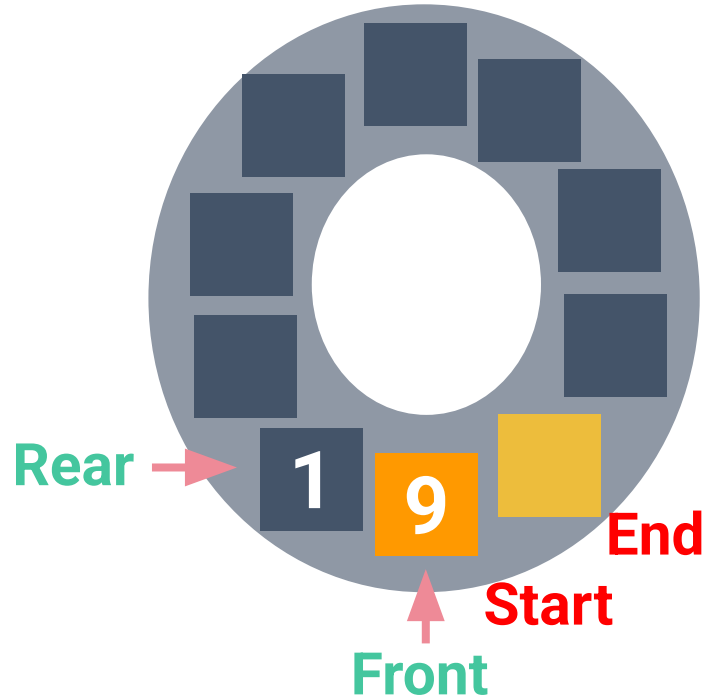
Front + Rear

Circular Queue

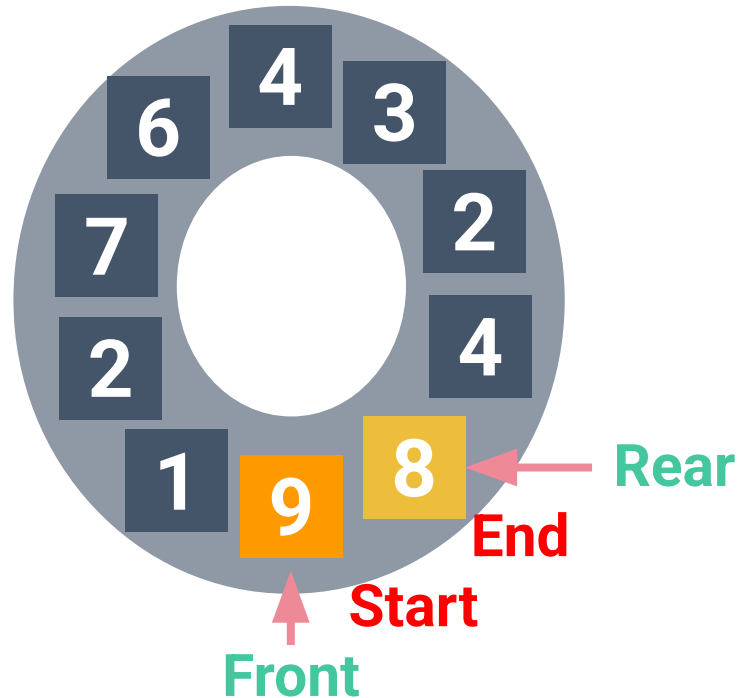




Circular Queue



Circular Queue

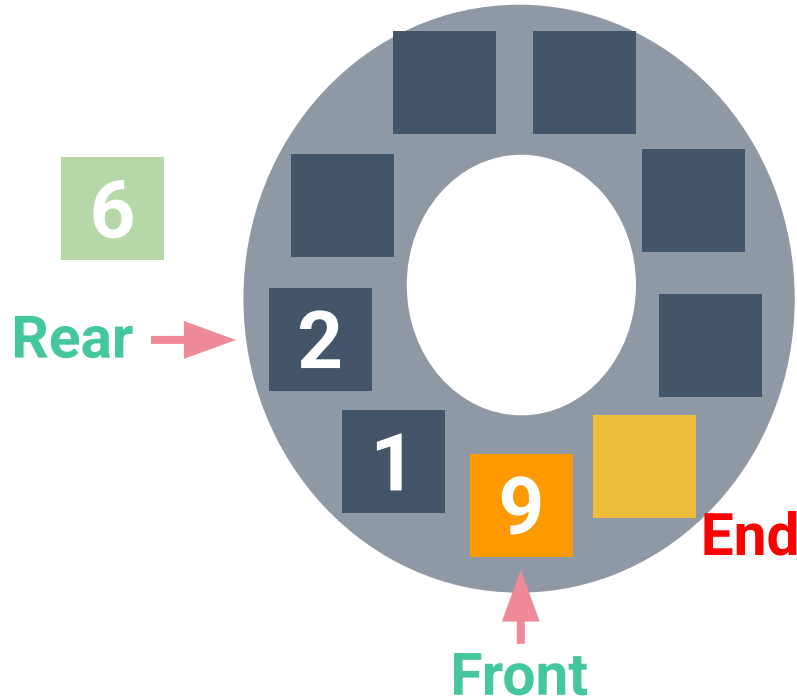




Circular Queue



Enqueue

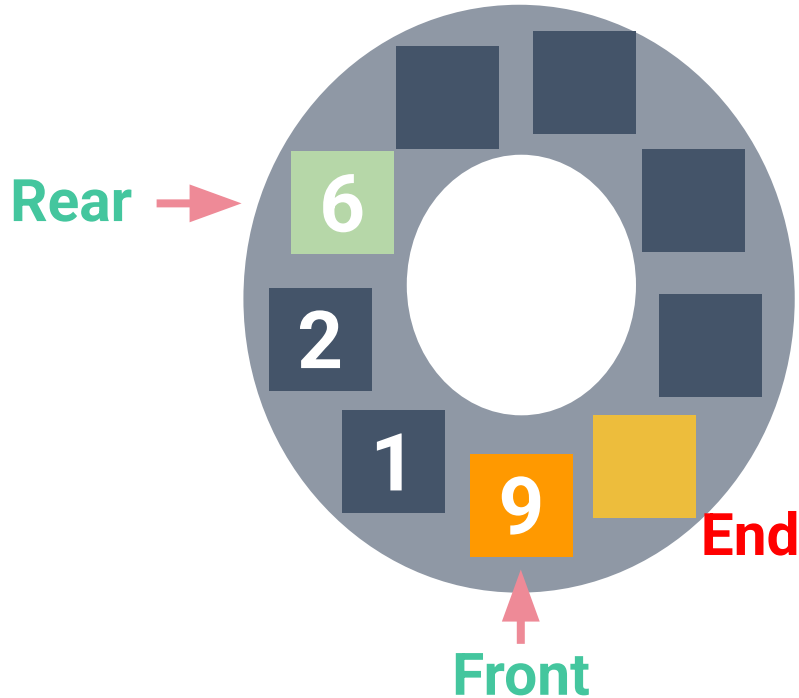




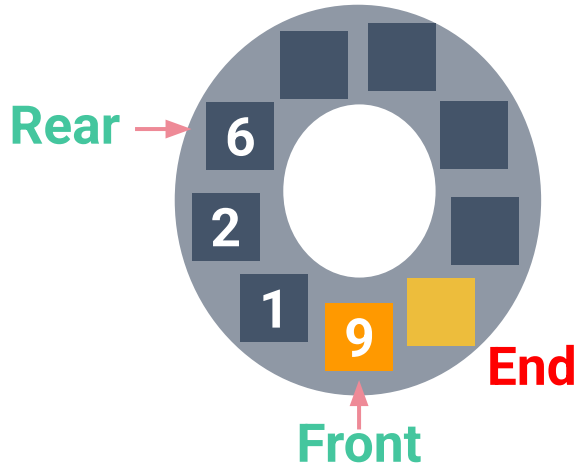
Circular Queue



Enqueue



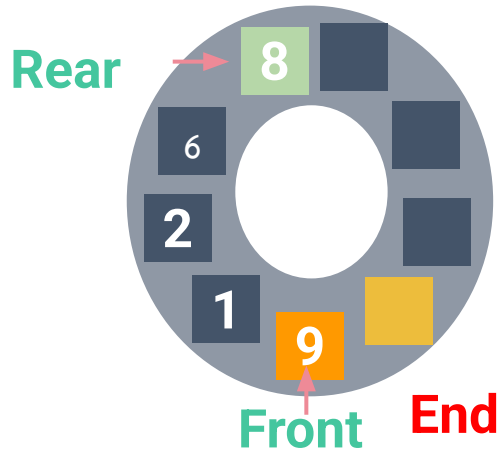
Circular Queue in Python



When **enqueue**, the rear pointer is given by

$$r = (r + 1) \% N$$

where N is the array length.



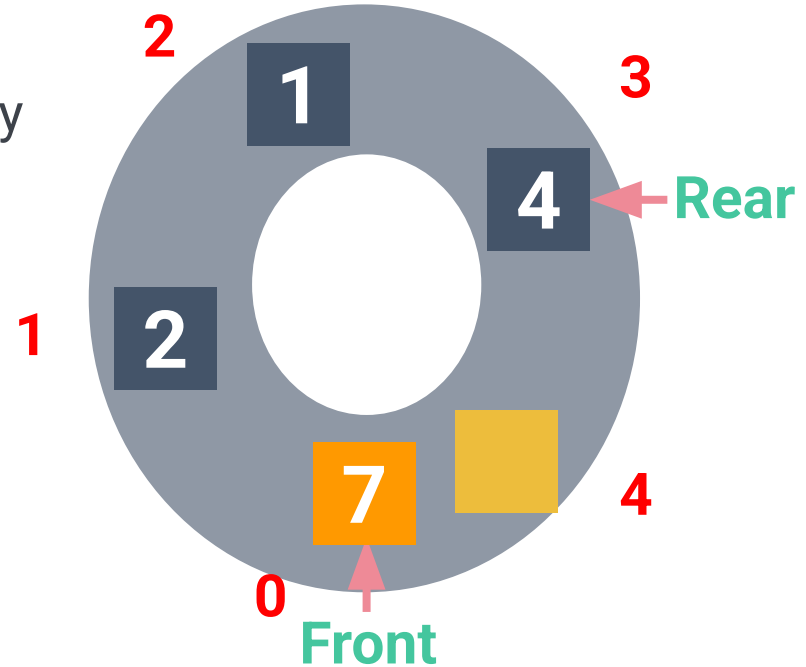
* Note: % denotes **modulo** operation in python which provides the remainder after division.

Circular Queue in Python

When **enqueue**, the rear pointer is given by

$$r = (r + 1) \% N$$

where N is the array length.



For example, given a list of length 5, current rear pointer is at 3.

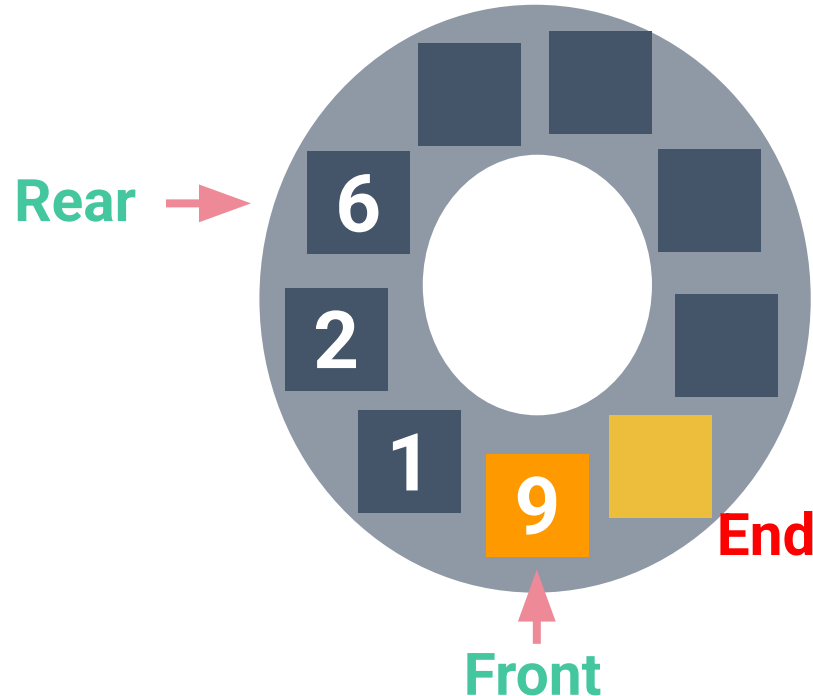
When an enqueue is called, the new index for rear pointer is $(3+1) \% 5 = 4$.



Circular Queue



Dequeue

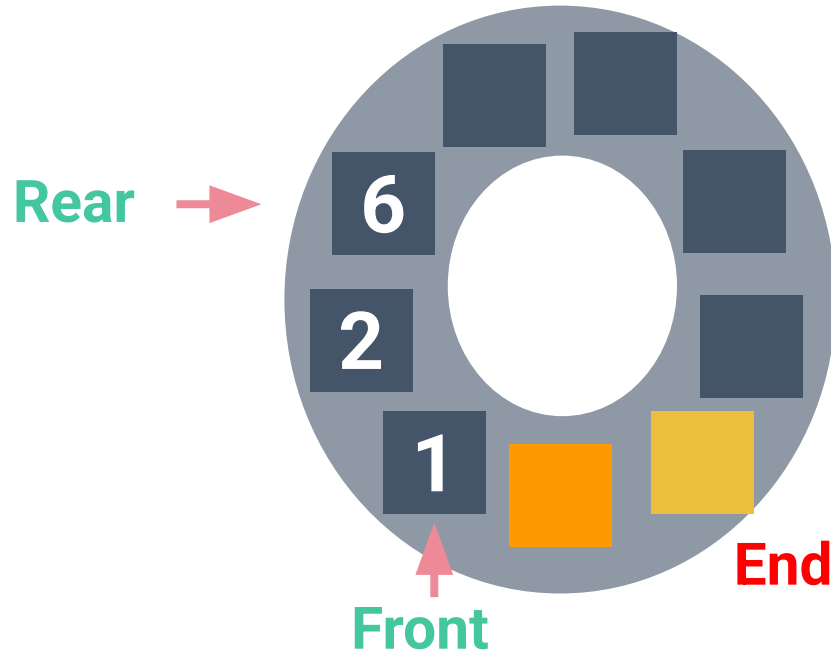




Circular Queue

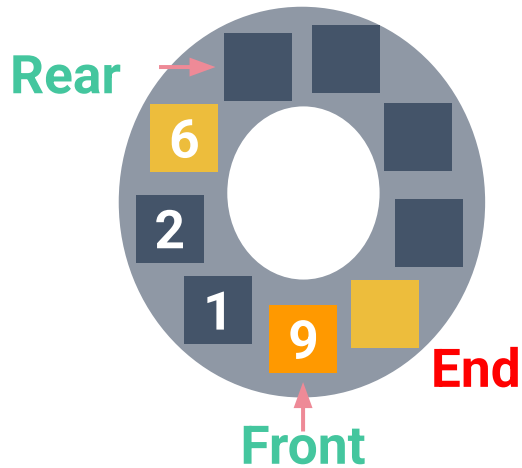


Dequeue





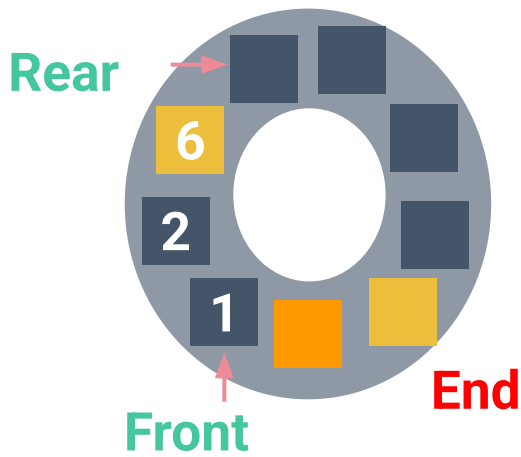
Circular Queue in Python



When **dequeue**, the front pointer is given by

$$f = (f + 1) \% N$$

where N is the array length.





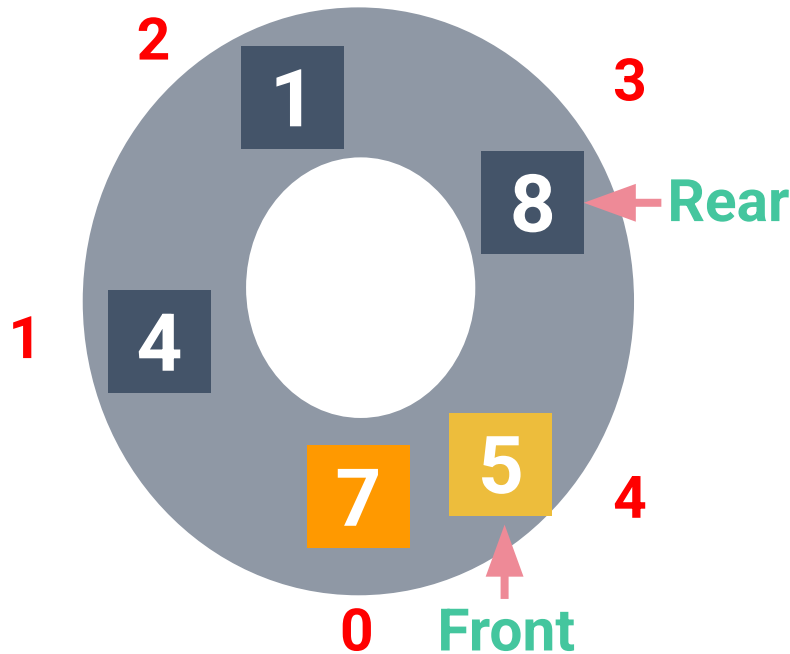
Circular Queue in Python



When **dequeue**, the front index is given by

$$f = (f + 1) \% N$$

where N is the array length.



For example, given a list of length 5, current front pointer is at 4.

When a dequeue is called, the new index for front pointer is $(4+1) \% 5 = 0$.